AWS Lab: Production Secrets Management & Credential Rotation Pipeline

Duration : 90 Min.

Scenario

A security audit uncovers a plaintext database password committed inside an application configuration file 'database.json' deployed to 15 EC2 instances. The database password must be rotated immediately. The cloud security engineer must: (1) create an encrypted secret in AWS Secrets Manager using a customer-managed KMS key, (2) grant an IAM role attached to the EC2 instances permission to read the secret, (3) rewrite the application bootstrap script to fetch the credential dynamically at runtime, and (4) verify that the plaintext configuration file is purged from disk.

Input Details

The cloud environment has been pre-configured with the following parameters: - **Platform:** AWS Secrets Manager & IAM Credential Hygiene - **Hardcoded Config:** `/opt/app_config/database.json` (Plaintext credentials) - **Key Management:** AWS KMS customer-managed key encryption

Username & Naming Conventions

Your candidate prefix is: **`<prefix>`** (derived from your login email before the `@` symbol, e.g. `student-demo01`). All custom resources and cloud configurations should incorporate this convention: - **Resource Naming:** Prepend or append `<prefix>` to your created AWS resources (e.g. `<prefix>-resource`). - **Candidate Tag:** Add tag `CandidatePrefix` = `<prefix>` on all managed cloud resources.

Task Objectives

Perform the following technical objectives directly in the AWS Management Console or using the AWS CLI: 1. Ingest exposed configuration file /opt/app_config/database.json containing plaintext credentials. 2. Create a secret in AWS Secrets Manager: aws secretsmanager create-secret --name prod/db/credentials --secret-string '{"username":"db_admin","password":"NewStrongP@ss2026"}'. 3. Attach an IAM policy granting secretsmanager:GetSecretValue to the instance profile role. 4. Update the application bootstrap script ~/bootstrap.py to retrieve the password via AWS SDK (boto3) at startup. 5. Securely wipe the plaintext configuration file from disk. 6. Verify that the application connects to the database successfully using the dynamic credential. 7. Generate a comprehensive audit sign-off report at ~/secrets_rotation_signoff.json.

Verification

Once completed, run the verification script to check your progress and receive your score. The verification system will inspect the live resources in the AWS cloud:

```
python run.py
```

Grading Criteria

Your performance will be evaluated based on the following test cases:

Test Case	Requirement	Marks
TC1	Secret creation: create-secret command correctly encapsulates username/password JSON	3 Marks
TC2	KMS encryption: secret is encrypted using specified KMS customer-managed key	3 Marks
TC3	IAM role scoping: IAM policy grants least-privilege GetSecretValue restricted to the secret ARN	3 Marks
TC4	SDK integration: bootstrap.py correctly queries Secrets Manager using boto3 without hardcoded keys	3 Marks
TC5	Artifact sanitization: plaintext password completely removed from local filesystem	3 Marks
TC6	Database authentication: verify successful connection establishment using the retrieved secret	3 Marks
TC7	Audit report: secrets_rotation_signoff.json documents rotation timestamp, KMS ARN, and verification	2 Marks

Total Score: 20 Marks

Important Notes

- Perform all actions directly in the AWS Management Console or using the AWS CLI.
- Ensure all resource configurations, tags, and naming conventions match the requirements.
- Do not terminate or alter pre-existing cloud infrastructure outside the scope of this assessment.